

COMPLEMENTI DI RETI

Manuale completo di Kathara

Teoria, comandi, struttura dei laboratori, procedure di verifica e oltre 200 domande per l'orale

KATHARA
topologie e workflow

ROUTING
RIP, OSPF e BGP

TRASPORTO
TCP, QUIC e congestione

Percorso di studio: da zero alla discussione orale

- Prima comprendi il modello mentale della rete e la struttura di un laboratorio.
- Poi esegui i laboratori seguendo un metodo di osservazione ripetibile.
- Infine usa le domande difficili per verificare la capacità di spiegare cause, effetti e guasti.

Edizione luglio 2026

Elaborato a partire dal materiale del corso e dalle relazioni di laboratorio fornite.

Come usare questo manuale

Questo documento è pensato come guida unica per tre attività: imparare i concetti, svolgere i laboratori Kathara senza procedere per tentativi casuali e allenarsi alla spiegazione orale. La struttura è volutamente progressiva: le prime parti costruiscono il linguaggio e gli strumenti; le parti centrali affrontano i protocolli; le ultime raccolgono playbook di laboratorio, checklist e domande.

Fase	Cosa fare	Criterio per passare alla fase successiva
1. Lettura attiva	Leggi un capitolo e riscrivi a parole tue i concetti chiave.	Sai distinguere entità, stato, messaggi e tabelle coinvolte.
2. Laboratorio	Avvia la topologia, esegui i comandi di osservazione prima e dopo la configurazione.	Sai prevedere l'output prima di eseguire il comando.
3. Guasto controllato	Spegni un link, rimuovi una rotta, cambia un costo o applica perdita/ritardo.	Sai spiegare perché il protocollo converge o fallisce.
4. Orale simulato	Rispondi alle domande senza terminale e senza appunti.	Risposta strutturata in 2-4 minuti con un esempio concreto.

Regola d'oro Ogni volta che modifichi la configurazione, osserva almeno: interfacce, indirizzi, tabella di routing, tabella del protocollo, vicini/adiacenze e traffico sul link. L'esame orale premia la capacità di collegare questi livelli.

Indice ragionato

Parte I - Fondamenti e strumenti

- Modello mentale delle reti
- Docker
- Architettura Kathara
- File di un laboratorio
- Comandi Kathara
- Comandi Linux di rete
- FRRouting e vtysh
- Metodo di troubleshooting

Parte II - Routing e indirizzamento

- IPv4, subnetting e routing statico
- IPv6, NDP e ARP
- RIP
- OSPF single-area
- OSPF multi-area
- BGP
- Policy e multihoming
- Data center e ECMP

Parte III - Misure, servizi e trasporto

- iperf3 e Linux Traffic Control

- Load balancing DNS/L4
- KaaS
- Trasporto affidabile e TCP
- Controllo di congestione e fairness
- QUIC
- SDN e gestione

Parte IV - Laboratori e orale

- Playbook dei laboratori del corso
- Troubleshooting per sintomo
- Simulazioni di orale
- Cheat sheet e template

PARTE I

Fondamenti e strumenti

Il modello mentale della rete, Docker, Kathara, Linux networking e FRRouting.

Obiettivo della parte Costruire una comprensione operativa: non memorizzare soltanto i comandi, ma saper prevedere cosa cambia nelle tabelle, nei pacchetti e nello stato dei protocolli.

1. Il modello mentale: forwarding, routing e piani della rete

Il livello rete nasce per interconnettere reti eterogenee tramite un sistema di indirizzamento comune e dispositivi intermedi. Per ragionare correttamente sui laboratori bisogna separare sempre il processo locale di inoltramento dal processo distribuito che costruisce le informazioni usate per inoltrare.

Concetto	Domanda a cui risponde	Dove avviene	Struttura dati
Forwarding / inoltramento	Dove invio questo singolo pacchetto?	Localmente su ogni host/router, per pacchetto	FIB / tabella di inoltramento
Routing / instradamento	Quali percorsi devono essere installati?	Nel control plane, staticamente o tramite protocolli	RIB, LSDB, BGP table, ecc.
Data plane	Come passa il traffico?	Kernel/switching hardware	FIB, neighbor table, code
Control plane	Come si apprendono e selezionano le rotte?	Processi e protocolli di controllo	RIB e database di protocollo

Un host esegue comunque un routing elementare: decide se una destinazione è on-link oppure remota. Se è on-link risolve il MAC del destinatario e consegna direttamente; se è remota consegna il frame al default gateway. Un router svolge la stessa decisione per ogni pacchetto ricevuto e può avere più interfacce e più rotte alternative.

Longest Prefix Match Se più rotte corrispondono alla destinazione, vince la rotta con il prefisso più lungo, cioè più specifico. La default route /0 è l'ultima possibilità, non la prima.

```
# Esempio concettuale di tabella
10.0.0.0/8      via 192.0.2.1
10.10.0.0/16   via 192.0.2.2
10.10.4.0/24   via 192.0.2.3
default        via 192.0.2.254

# Destinazione 10.10.4.8 -> viene scelta 10.10.4.0/24
```

Nel control plane tradizionale ogni router calcola le proprie rotte in modo distribuito. Nel modello SDN, invece, un controller logicamente centralizzato mantiene una visione globale e programma gli apparati tramite API southbound.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Spiega la differenza tra forwarding e routing usando il viaggio di un singolo pacchetto come esempio.
2. **[Base]** Perché anche un normale host possiede una tabella di routing?
3. **[Intermedia]** Qual è la differenza tra RIB e FIB e perché possono non contenere esattamente le stesse rotte?
4. **[Intermedia]** Che cosa succede quando una destinazione è nella stessa subnet del mittente? Quale ruolo ha ARP?
5. **[Intermedia]** Che cosa cambia quando la destinazione è remota? Quale MAC viene inserito nel frame Ethernet?

6. **[Difficile]** Una tabella contiene 10.0.0.0/8, 10.10.0.0/16 e 0.0.0.0/0. Quale rotta viene scelta per 10.10.20.1 e perché?
7. **[Difficile]** Perché un protocollo di routing può conoscere una rotta che il kernel non installa nella FIB?
8. **[Difficile]** Confronta control plane distribuito e SDN dal punto di vista di scalabilità, failure domain e latenza di reazione.
9. **[Molto difficile]** Descrivi il percorso completo di un pacchetto da un processo applicativo a un host remoto, indicando socket, routing lookup, neighbor lookup e inoltra sui router.

2. Docker: il motore sottostante

Kathara usa container per emulare i dispositivi. Un container è un insieme di processi isolati che condivide il kernel dell'host ma dispone di namespace propri per rete, processi, mount e altre risorse; i cgroup limitano e contabilizzano CPU e memoria. Una macchina virtuale, invece, virtualizza l'hardware e avvia un kernel guest completo.

Oggetto	Natura	Persistenza	Analogia utile
Immagine	Template a livelli, di sola lettura	Rimane sul disco finché non viene rimossa	Classe / eseguibile
Container	Istanza creata da un'immagine con layer scrivibile	Può essere in esecuzione o fermo	Oggetto / processo
Volume / bind mount	Dati montati dall'esterno	Persistono indipendentemente dal container	Disco dati
Network Docker	Dominio L2/L3 virtuale	Gestito dal daemon Docker	Switch/segmento virtuale

Comando	Scopo	Nota
docker run hello-world	Verifica l'installazione e avvia un container di test	Esegue implicitamente pull se l'immagine manca.
docker pull IMAGE	Scarica un'immagine	Non crea alcun container.
docker images	Elenca le immagini locali	Mostra repository, tag, ID e dimensione.
docker ps	Elenca i container in esecuzione	Con -a include anche quelli terminati.
docker logs CONTAINER	Mostra stdout/stderr del container	Utile nel debugging dei servizi.
docker exec -it CONTAINER sh	Apri una shell in un container esistente	Per i dispositivi Kathara preferire kathara connect.
docker stop CONTAINER	Arresta un container	Invia prima un segnale di terminazione.
docker rm CONTAINER	Rimuove un container	Non rimuove l'immagine.
docker rmi IMAGE	Rimuove un'immagine	Può fallire se esistono container dipendenti.
docker container prune	Rimuove tutti i container fermi	Operazione distruttiva: leggere la conferma.
docker system df	Mostra l'uso disco di Docker	Utile prima di fare pulizia.

Comando	Scopo	Nota
docker system prune	Pulisce risorse inutilizzate	Usare con attenzione, specialmente con -a.

Attenzione Durante un laboratorio non rimuovere manualmente container o network con Docker salvo che tu stia diagnosticando un problema. Kathara mantiene una propria relazione tra scenario, device e collision domain: usa prima lclean o wipe.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Qual è la differenza tra immagine e container?
2. **[Base]** Perché docker run può funzionare anche se non hai eseguito docker pull?
3. **[Base]** Che differenza c'è tra docker ps e docker ps -a?
4. **[Intermedia]** Perché i container sono più leggeri delle VM e quale componente condividono?
5. **[Intermedia]** Quale ruolo hanno namespace e cgroup?
6. **[Intermedia]** Perché la rimozione di un container non implica la rimozione dell'immagine?
7. **[Difficile]** Quali vantaggi e limiti introduce la condivisione del kernel rispetto all'isolamento di una VM?
8. **[Difficile]** Come collegheresti il concetto di network namespace al fatto che ogni device Kathara vede proprie interfacce eth0, eth1, ...?
9. **[Molto difficile]** Descrivi cosa accade internamente quando Kathara avvia due device sullo stesso collision domain, facendo riferimento a container, interfacce virtuali e rete Docker.

3. Architettura di Kathara e logica di funzionamento

Kathara è un emulatore di rete container-based. Ogni dispositivo è un container Linux; ogni collegamento L2 è rappresentato da un collision domain virtuale. Collegare eth0 di due device allo stesso dominio equivale a collegare due schede allo stesso segmento Ethernet. Kathara può usare Docker per l'esecuzione locale e, in scenari distribuiti, un backend basato su Kubernetes/Megalos.

Livello	Oggetto Kathara	Oggetto reale simulato
Device	Container con filesystem, processi e network namespace	Host, router, server, switch software
Interfaccia	eth0, eth1, ... nel namespace del device	Scheda/porta di rete
Collision domain	Rete virtuale condivisa da più interfacce	LAN, cavo p2p, segmento Ethernet
Image	kathara/base, kathara/frr o immagine custom	Sistema operativo e software del dispositivo
Scenario	Cartella con lab.conf, script e file	Intera infrastruttura di laboratorio

Modello mentale dell'emulazione

Host fisico

```
|-- Docker / backend Kathara
|  |-- container pc1 -- eth0 ----+
|  |                               +---- collision domain A
|  |-- container pc2 -- eth0 ----+
|  |
|  |-- container r1  -- eth0 ---- collision domain B
|  |                  -- eth1 ---- collision domain C
```

Kathara non configura automaticamente indirizzi IP o protocolli di routing solo perché due interfacce sono collegate. Il file `lab.conf` definisce la topologia; gli script `.startup` e i file copiati nel filesystem configurano il comportamento dei device. Questa separazione è fondamentale: livello 2, indirizzamento, forwarding e control plane sono passaggi distinti.

Domanda tipica d'orale “Il ping non funziona: il `lab.conf` è corretto. Perché?” Risposta: `lab.conf` crea interfacce e domini L2; non assegna necessariamente IP, non crea rotte remote e non abilita il forwarding IP. Devi verificare ogni livello.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Che cosa rappresentano un device e un collision domain in Kathara?
2. **[Base]** Perché due device collegati allo stesso collision domain non comunicano necessariamente via IP?
3. **[Intermedia]** Quale isolamento offre il network namespace a un device Kathara?
4. **[Intermedia]** Perché la latenza di ping tra container può variare anche in una topologia virtuale molto semplice?
5. **[Difficile]** Distingui fedeltà funzionale ed equivalenza prestazionale in un emulatore container-based.
6. **[Difficile]** Perché un collision domain è un concetto L2 mentre una subnet è un concetto L3? Possono non coincidere?
7. **[Molto difficile]** Progetta una topologia con due LAN e un router: indica collision domain, interfacce, subnet e condizioni necessarie perché il traffico end-to-end funzioni.

4. Struttura di un laboratorio: file, cartelle e ordine di esecuzione

Uno scenario è una cartella autosufficiente. Il nome della cartella non determina la topologia; sono i file e le sottocartelle a descrivere device, collegamenti e configurazioni. La struttura seguente è il riferimento da imparare.

```
mio-lab/
|-- lab.conf
|-- lab.dep           # opzionale: dipendenze di avvio
|-- lab.ext           # opzionale: domini verso rete fisica (Linux/root)
|-- shared/           # montata come /shared in tutti i device
|-- shared.startup     # eseguito su ogni device prima di device.startup
|-- shared.shutdown   # eseguito dopo device.shutdown
|-- pc1.startup
|-- pc1.shutdown
|-- pc1/              # contenuto copiato nella root / di pc1
|  |-- etc/hosts
|-- r1.startup
|-- r1/
|  |-- etc/frr/
```



```
| -- daemons
|-- frr.conf
```

Componente	Funzione	Quando serve
lab.conf	Definisce device, interfacce, collision domain, immagini e opzioni	Sempre, salvo casi avanzati forzati.
device.startup	Script shell eseguito all'avvio del singolo device	IP, rotte, sysctl, avvio servizi.
device.shutdown	Script shell eseguito quando il device viene arrestato	Pulizia, salvataggi, chiusure controllate.
device/	Gerarchia copiata nel filesystem del device a partire da /	Configurazioni persistenti di FRR, DNS, web server, script.
shared/	Cartella host condivisa come /shared	Catture, chiavi TLS, risultati iperf, log, script.
shared.startup/.shutdown	Script comune a tutti i device	Configurazione ripetuta o preparazione comune.
lab.dep	Vincoli di avvio: device: dipendenza1 dipendenza2	Quando un servizio deve essere pronto prima di un altro.
lab.ext	Mappa collision domain su interfacce fisiche/VLAN	Integrazione con reti esterne, solo Linux con privilegi.

Il file lab.conf usa assegnazioni del tipo device[arg]="valore". Se arg è un numero, indica l'indice dell'interfaccia; il valore è il collision domain. Se arg è un nome, imposta un'opzione del device.

Esempio di lab.conf

```
LAB_NAME="K2-TwoHosts"
LAB_DESCRIPTION="Due host sullo stesso segmento"

pc1[0]="A"
pc2[0]="A"

# Opzioni possibili
pc1[image]="kathara/base"
pc1[mem]="128m"
pc1[cpus]="0.5"
pc1[port]="8080:80/tcp"
pc1[bridged]="true"
pc1[sysctl]="net.ipv4.ip_forward=1"
pc1[env]="MODE=test"
```

Una sottocartella device/ viene interpretata come una root filesystem overlay. Il file r1/etc/frr/frr.conf, per esempio, viene copiato in /etc/frr/frr.conf dentro r1. Questa è la tecnica principale per consegnare configurazioni ai demoni di rete.

```
# pc1.startup
ip link set eth0 up
ip address add 10.0.0.1/24 dev eth0

# r1.startup
ip link set eth0 up
ip link set eth1 up
ip address add 192.168.1.254/24 dev eth0
ip address add 192.168.2.254/24 dev eth1
sysctl -w net.ipv4.ip_forward=1
```

Ordine logico Prima viene preparato il filesystem, poi vengono eseguiti gli script comuni e infine lo startup specifico del device. Non affidarti a ritardi casuali: quando esistono dipendenze reali, usa lab.dep o controlli di readiness negli script.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Che differenza c'è tra lab.conf e pc1.startup?
2. **[Base]** Dove viene montata la cartella shared dentro i device?
3. **[Intermedia]** Come fai a far comparire un file locale in /etc/frr/frr.conf dentro r1?
4. **[Intermedia]** A cosa serve lab.dep e quale sintassi usa?
5. **[Intermedia]** Perché device.startup non deve necessariamente avere il bit eseguibile?
6. **[Difficile]** Che differenza c'è tra copiare un file nella directory device/ e montare un volume con l'opzione volume?
7. **[Difficile]** Descrivi una situazione in cui shared.startup è preferibile a ripetere lo stesso comando in ogni .startup.
8. **[Difficile]** Quali rischi introduce lab.ext e perché richiede privilegi elevati?
9. **[Molto difficile]** Progetta la struttura di cartelle per tre router FRR e due host, indicando esattamente dove collocheresti daemons, frr.conf, script di test e catture.

5. Comandi Kathara: ciclo di vita, accesso e diagnostica

Comando	Uso operativo	Esempio
kathara check	Verifica ambiente, backend, immagine e versioni	kathara check
kathara settings	Mostra/modifica la configurazione utente	kathara settings
kathara lstart	Avvia lo scenario nella cartella corrente	kathara lstart --noterminals
kathara lstart -d DIR	Avvia uno scenario da un percorso esplicito	kathara lstart -d ./K4-BGP
kathara lstart --print	Dry-run del lab.conf	kathara lstart --print
kathara lstart DEVICE...	Avvia solo alcuni device	kathara lstart r1 r2
kathara lstart --exclude D...	Avvia tutti tranne alcuni device	kathara lstart --exclude ws3
kathara linfo	Stato dettagliato dei device del lab corrente	kathara linfo
kathara linfo -t	Mappa device-collision domain	kathara linfo -t
kathara linfo -w	Monitoraggio live	kathara linfo -w
kathara list	Elenca tutti i device Kathara dell'utente	kathara list -w
kathara connect DEVICE	Apri una shell nel device	kathara connect r1
kathara connect -l DEVICE	Mostra log di startup e poi apri la shell	kathara connect -l r1

Comando	Uso operativo	Esempio
kathara exec DEVICE CMD	Esegue un comando senza shell interattiva	kathara exec r1 "ip route"
kathara exec --wait ...	Attende la fine degli startup prima del comando	kathara exec --wait r1 "vtysh -c 'show ip route'"
kathara lrestart	Esegue lclean seguito da lstart	kathara lrestart --noterminals
kathara lclean	Arresta e rimuove lo scenario corrente	kathara lclean
kathara wipe -f	Rimuove tutti i device/domini Kathara dell'utente	kathara wipe -f
kathara lconfig	Aggiunge/rimuove interfacce a un device in esecuzione	kathara lconfig -n pc1 --add X
kathara vstart/vclean	Gestisce device singoli fuori da uno scenario	kathara vstart -n pc1 --eth 0:A

```
# Workflow consigliato
cd ~/labs/K2-TwoHosts
kathara lstart --print
kathara lstart --noterminals
kathara linfo -t
kathara exec --wait pc1 "ip -br address"
kathara exec pc1 "ping -c 2 10.0.0.2"
kathara connect pc1
kathara lclean
```

Perché --noterminals? Su desktop Linux, macOS o Windows l'apertura automatica di molte finestre può essere scomoda o fallire. Avviare senza terminali e usare connect/exec rende i test più riproducibili e adatti agli script.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Quali comandi useresti per avviare, ispezionare e arrestare un laboratorio?
2. **[Base]** Che differenza c'è tra kathara linfo e kathara list?
3. **[Intermedia]** Perché kathara exec è utile nei test automatici?
4. **[Intermedia]** A cosa serve l'opzione --wait di kathara exec?
5. **[Intermedia]** Quando useresti lstart -d invece di cambiare directory?
6. **[Difficile]** Qual è la differenza concettuale tra lstart e vstart?
7. **[Difficile]** Che cosa fa realmente lrestart e quali effetti ha sullo stato runtime non salvato?
8. **[Difficile]** Perché wipe è più drastico di lclean?
9. **[Molto difficile]** Scrivi una procedura non interattiva che avvii un lab, attenda gli startup, verifichi tre proprietà di rete, salvi gli output e pulisca sempre lo scenario anche in caso di errore.

6. Comandi Linux di rete: la cassetta degli attrezzi

Obiettivo	Comando moderno	Interpretazione
Vista compatta delle interfacce	<code>ip -br link</code>	Stato UP/DOWN e nomi.
Indirizzi IPv4/IPv6	<code>ip -4 addr show / ip -6 addr show</code>	Prefissi, scope e stato.
Aggiungere un indirizzo	<code>ip addr add 10.0.0.1/24 dev eth0</code>	Crea anche la rotta connessa.
Rimuovere un indirizzo	<code>ip addr del 10.0.0.1/24 dev eth0</code>	Può eliminare la rotta connessa associata.
Attivare un'interfaccia	<code>ip link set eth0 up</code>	Abilita il link dal punto di vista amministrativo.
Tabella di routing	<code>ip route show</code>	Rotte connesse, statiche e dinamiche installate.
Aggiungere default route	<code>ip route add default via 10.0.0.254</code>	Next-hop deve essere raggiungibile.
Aggiungere rotta specifica	<code>ip route add 10.20.0.0/16 via 10.0.0.2</code>	Usata secondo LPM e metrica.
Calcolare la rotta effettiva	<code>ip route get 10.20.1.5</code>	Mostra next-hop, interfaccia e source scelti.
Neighbor/ARP/NDP	<code>ip neigh show</code>	Associazioni IP-MAC e stato.
Svuotare neighbor cache	<code>ip neigh flush all</code>	Forza nuove risoluzioni.
Ping IPv4/IPv6	<code>ping -c 3 IP / ping -6 -c 3 IP</code>	Connettività e RTT, non prova ogni servizio.
Percorso	<code>traceroute IP</code>	Inferisce i router tramite TTL/Hop Limit.
Socket in ascolto	<code>ss -ltnup</code>	Porte TCP/UDP e processi.
Cattura	<code>tcpdump -ni eth0</code>	Osservazione pacchetti senza risoluzione nomi.
Forwarding IPv4	<code>sysctl net.ipv4.ip_forward</code>	1 abilita l'inoltro tra interfacce.

```
# Diagnosi minima su un host
ip -br link
ip -br address
ip route
ip route get 10.20.1.10
ip neigh show
ping -c 2 10.20.1.10
traceroute -n 10.20.1.10

# Diagnosi su un router
sysctl net.ipv4.ip_forward
ip route
ss -ltnup
tcpdump -ni any icmp
```

Impara a leggere gli errori: “Network is unreachable” indica che il kernel non ha trovato una rotta utilizzabile; “Destination Host Unreachable” spesso indica fallimento ARP/neighbor o un ICMP generato da un router; un timeout puro può dipendere da perdita, firewall, assenza di rotta di ritorno o servizio non in ascolto.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Quale comando mostra indirizzi e prefissi in modo compatto?
2. **[Base]** Quale rotta viene creata automaticamente quando assegni 10.0.0.1/24 a eth0?
3. **[Intermedia]** Che differenza c'è tra ip route show e ip route get DEST?
4. **[Intermedia]** Perché un next-hop deve essere raggiungibile tramite una rotta connessa o ricorsiva?
5. **[Intermedia]** Come distingueresti un problema di routing da un problema ARP usando ip route e ip neigh?
6. **[Difficile]** Un host ha due default route con metriche diverse. Come prevedi quella scelta e come la verifichi?
7. **[Difficile]** Perché ping può funzionare mentre una connessione TCP fallisce?
8. **[Difficile]** Che informazioni aggiunge tcpdump rispetto a ping e traceroute?
9. **[Molto difficile]** Descrivi una sequenza di comandi per localizzare un guasto end-to-end senza modificare la configurazione.

7. FRRouting e vtysh

FRRouting (FRR) trasforma un sistema Linux in un router software. L'architettura è modulare: daemons come ripd, ospfd e bgpd implementano i protocolli; zebra raccoglie le rotte e dialoga con il kernel. vtysh fornisce una CLI unificata in stile apparati di rete.

```
/etc/frr/
|-- daemons      # abilita/disabilita i processi: ripd=yes, ospfd=yes, bgpd=yes
-- frr.conf       # configurazione integrata

# Nel lab Kathara:
r1/etc/frr/daemons -> /etc/frr/daemons nel container
r1/etc/frr/frr.conf -> /etc/frr/frr.conf nel container
```

Modalità vtysh	Prompt tipico	Cosa puoi fare
User	r1>	Comandi show e diagnostica di base.
Privileged	r1#	Informazioni avanzate, clear, accesso alla configurazione.
Global config	r1(config)#	Modifica persistente della configurazione.
Protocol/interface	r1(config-router)# / (config-if)#	Comandi specifici del contesto.

```
vtysh
show running-config
show ip route
show daemons

configure terminal
hostname r1
router ospf
  ospf router-id 1.1.1.1
exit
write memory

# Per rimuovere una configurazione:
configure terminal
no router ospf
```

La CLI è operativa: “no” nega o rimuove un comando già presente. Non confondere la tabella di un protocollo con la tabella globale. Per esempio `show ip bgp` mostra la BGP RIB, mentre `show ip route` mostra la RIB integrata di FRR e il kernel espone la FIB tramite `ip route`.

Vista	Comando	Domanda
Kernel/FIB	<code>ip route</code>	Quali rotte usa effettivamente Linux per inoltrare?
FRR globale	<code>vttysh -c "show ip route"</code>	Quali rotte ha selezionato FRR tra le sorgenti?
OSPF	<code>show ip ospf route / database</code>	Cosa ha calcolato OSPF e con quale LSDB?
BGP	<code>show ip bgp / summary</code>	Quali NLRI e path sono noti? Quali peer sono attivi?

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Quali ruoli hanno zebra e i daemon specifici di protocollo?
2. **[Base]** Come entri nella modalità di configurazione di `vttysh`?
3. **[Intermedia]** Perché il comando `no` è importante in una configurazione operativa?
4. **[Intermedia]** Che differenza c'è tra `show ip route` eseguito in `vttysh` e `ip route` eseguito nella shell Linux?
5. **[Intermedia]** A cosa serve il file `/etc/frr/daemons`?
6. **[Difficile]** Descrivi il percorso di una rotta da `ospfd` fino alla FIB del kernel.
7. **[Difficile]** Perché una rotta presente in `show ip bgp` può non comparire in `ip route`?
8. **[Molto difficile]** Spiega come isoleresti un problema tra protocollo, zebra e kernel usando solo comandi `show` e `ip route`.

8. Metodo sistematico di troubleshooting

La diagnosi efficace procede dal basso verso l'alto e dal locale verso il remoto. Non iniziare modificando rotte o riavviando tutto: prima raccogli evidenze. Ogni cambiamento non motivato può nascondere il guasto originale.

Passo	Verifica	Comandi
1. Stato del lab	Device avviati e collegamenti previsti	<code>kathara linfo -t</code> ; <code>kathara list</code>
2. Link	Interfacce presenti e UP	<code>ip -br link</code>
3. Indirizzi	IP, prefissi, duplicati, scope	<code>ip -br addr</code> ; <code>ip -6 addr</code>
4. On-link	Neighbor resolution e ping al next-hop	<code>ip neigh</code> ; <code>ping gateway</code>
5. Routing locale	Rotta scelta, next-hop, source	<code>ip route</code> ; <code>ip route get DEST</code>
6. Forwarding	Router abilitato e rotte di ritorno	<code>sysctl</code> ; <code>ip route</code> su ogni hop

Passo	Verifica	Comandi
7. Control plane	Vicini, database, annunci, filtri	show ip ospf neighbor; show ip bgp summary
8. Trasporto/app	Socket e servizio	ss -lntup; curl; links
9. Pacchetti	Dove si interrompe il flusso	tcpdump su interfacce successive

Rotta di ritorno Il ping richiede un percorso in entrambe le direzioni. Un errore comune è configurare correttamente l'andata e dimenticare il ritorno. La connettività è una proprietà end-to-end, non del solo mittente.

Sintomo	Ipotesi prioritarie
Network is unreachable	Nessuna rotta; interfaccia senza IP; default route mancante.
Destination Host Unreachable dall'host locale	ARP/NDP fallito; host non sul link; interfaccia down.
Ping al gateway OK, remoto KO	Forwarding disabilitato; rotta remota o di ritorno mancante; firewall.
OSPF Init/2-Way ma non Full	Comportamento DR/DROther normale oppure mismatch di rete/parametri.
OSPF ExStart/Exchange	Spesso MTU mismatch o problemi di database exchange.
BGP Active	Connettività TCP 179 assente, remote-as errato, source/neighbor errato.
BGP Established ma zero prefissi	Nessun network/redistribute, filtri, import-check, route non presente localmente.
DNS risolve IP guasto	Cache e TTL; record non ancora scaduto.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

- [Base]** Qual è l'ordine minimo di verifiche quando un ping fallisce?
- [Intermedia]** Perché riavviare subito il lab è una cattiva pratica diagnostica?
- [Intermedia]** Come dimostri che il problema è una rotta di ritorno mancante?
- [Difficile]** Un pacchetto raggiunge il server ma la risposta non torna. Quali catture faresti e dove?
- [Difficile]** BGP è Established ma la LAN remota non è raggiungibile. Elenca almeno cinque verifiche in ordine.
- [Difficile]** OSPF non forma adiacenza. Quali parametri devono essere compatibili?
- [Molto difficile]** Costruisci un albero decisionale per distinguere guasto L2, L3, control plane e applicazione.

PARTE II

Routing e indirizzamento

IPv4/IPv6, ARP/NDP, routing statico, RIP, OSPF, BGP e data center.

Obiettivo della parte Costruire una comprensione operativa: non memorizzare soltanto i comandi, ma saper prevedere cosa cambia nelle tabelle, nei pacchetti e nello stato dei protocolli.

9. IPv4, subnetting, routing statico e routing loop

Un piano di indirizzamento assegna prefissi non sovrapposti alle reti e indirizzi alle interfacce. Il prefisso determina quali bit identificano la rete e quanti restano per gli host. La VLSM permette di usare maschere diverse per dimensionare ogni segmento in base ai requisiti.

Prefisso	Indirizzi totali	Host tradizionalmente utilizzabili	Uso tipico
/24	256	254	LAN piccola
/25	128	126	LAN fino a circa 100 host
/30	4	2	Link punto-punto IPv4 classico
/31	2	2 su link p2p compatibili	Punto-punto senza network/broadcast tradizionali
/32	1	1 indirizzo host/loopback	Route host, router-id/loopback

```
# Host A
ip addr add 192.168.1.1/24 dev eth0
ip route add default via 192.168.1.254

# Router r1
ip addr add 192.168.1.254/24 dev eth0
ip addr add 192.168.2.254/24 dev eth1
sysctl -w net.ipv4.ip_forward=1

# Host B
ip addr add 192.168.2.1/24 dev eth0
ip route add default via 192.168.2.254
```

Il routing statico è adatto a topologie piccole e deterministiche. Non genera traffico di controllo e rende esplicite le policy, ma non converge automaticamente in caso di guasto e diventa oneroso quando aumentano prefissi e percorsi alternativi.

In una topologia ad anello è possibile forzare percorsi asimmetrici configurando rotte diverse nei due sensi. L'asimmetria non è di per sé un errore: diventa problematica quando firewall stateful, NAT o sistemi di misura assumono simmetria. Un routing loop, invece, è un errore: due o più router si reinviano il pacchetto. Il TTL IPv4 o Hop Limit IPv6 evita la circolazione infinita e produce ICMP Time Exceeded.

```
# Osservare il percorso e il TTL
ping -c 1 10.0.0.129
traceroute -n 10.0.0.129

# Una risposta con TTL=61, se il mittente remoto parte da 64,
# suggerisce che la risposta ha attraversato 3 router.
```

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Quanti host tradizionali supporta una /22 e come lo calcoli?
2. **[Base]** Perché una rotta connessa compare automaticamente dopo ip addr add?
3. **[Intermedia]** Perché un host ha bisogno di una default route per raggiungere una subnet diversa?
4. **[Intermedia]** Quale maschera sceglieresti per 1000 host? Giustifica il numero di bit host.
5. **[Intermedia]** Perché un link punto-punto può usare /30 o /31?
6. **[Difficile]** Confronta una rotta statica specifica e una default route in presenza di LPM.
7. **[Difficile]** Come dimostri che andata e ritorno seguono percorsi diversi?
8. **[Difficile]** Perché il TTL risolve il sintomo di un loop ma non corregge la tabella di routing?
9. **[Molto difficile]** Progetta un piano VLSM per due LAN da 1000 host e un link p2p dentro 10.10.0.0/16, evitando sovrapposizioni.
10. **[Molto difficile]** Spiega come un loop statico può produrre ICMP Time Exceeded e come traceroute sfrutta lo stesso meccanismo.

10. IPv6, NDP e ARP

IPv6 usa indirizzi a 128 bit scritti in otto gruppi esadecimali. Gli zeri iniziali di ogni gruppo possono essere omessi; una sola sequenza contigua di gruppi zero può essere compressa con ::. IPv6 elimina il broadcast e usa unicast, multicast e anycast. Ogni interfaccia genera normalmente un indirizzo link-local fe80::/10, valido sul solo link.

```
2001:0db8:0000:0000:0000:ff00:0042:8329
-> 2001:db8::ff00:42:8329

ip -6 addr add 2001:db8:1::1/64 dev eth0
ip -6 route add default via 2001:db8:1::fe
ip -6 addr show
ip -6 route show
ping -6 -c 2 2001:db8:1::2
```

NDP sostituisce e amplia le funzioni di ARP usando messaggi ICMPv6 multicast. Gestisce neighbor discovery, router discovery, duplicate address detection e supporta SLAAC. In IPv4 ARP traduce un indirizzo IPv4 on-link nel MAC necessario per costruire il frame Ethernet.

Fase ARP	Destinazione L2	Contenuto essenziale
ARP Request	Broadcast ff:ff:ff:ff:ff:ff	Chi possiede IP X? Comunichi il proprio MAC.
ARP Reply	Unicast al richiedente	IP X è associato al MAC Y.
Cache	Locale	Associazione temporanea con stato e timeout.

```
# Forzare e osservare una nuova risoluzione ARP
ip neigh flush all
tcpdump -ttni eth0 arp
# in un altro terminale:
ping -c 1 10.0.0.2

# Tabella neighbor moderna
ip neigh show
```

Il Gratuitous ARP annuncia spontaneamente una associazione IP-MAC, utile dopo un cambio di indirizzo, in alta disponibilità e per rilevare duplicati. Il Proxy ARP fa rispondere un router per un indirizzo che si trova altrove, attirando il traffico per poi inoltrarlo.

IPv6 e NAT L'ampio spazio di indirizzamento riduce la necessità del NAT come tecnica di conservazione degli indirizzi. Ciò non significa che spariscano firewall, policy o meccanismi di traduzione in ogni rete; soprattutto, NAT non deve essere confuso con una misura di sicurezza.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Comprimi correttamente 2001:0db8:0000:0000:0000:0000:0001.
2. **[Base]** Qual è la differenza tra global unicast e link-local?
3. **[Base]** Perché ARP è necessario anche quando conosci già l'IP del destinatario?
4. **[Intermedia]** Perché l'ARP Request è broadcast e l'ARP Reply normalmente unicast?
5. **[Intermedia]** Che cosa contiene la neighbor cache e perché ha un timeout?
6. **[Intermedia]** Quali funzioni svolge NDP oltre alla risoluzione dell'indirizzo di livello 2?
7. **[Difficile]** Quale MAC viene risolto da un host che invia a una destinazione remota: quello del server o quello del gateway? Perché?
8. **[Difficile]** Come cattureresti e dimostreresti l'intera sequenza ARP -> ICMP?
9. **[Difficile]** Confronta broadcast ARP e multicast NDP dal punto di vista dell'impatto sui nodi non interessati.
10. **[Molto difficile]** Spiega perché un problema di neighbor resolution può produrre un errore di connettività pur avendo una tabella di routing corretta.

11. RIP: Distance Vector e count-to-infinity

RIP è un IGP distance-vector basato su Bellman-Ford. Ogni router conosce il costo verso i vicini e scambia periodicamente le proprie distanze. La metrica è il numero di hop; 15 è il massimo raggiungibile e 16 rappresenta infinito. RIPv2 usa aggiornamenti multicast verso 224.0.0.9.

Il vantaggio è la semplicità; i limiti sono convergenza lenta, metrica povera e rischio di routing loop transitori. Quando un link cade, una “cattiva notizia” può propagarsi lentamente: router vicini si convincono a vicenda che esista ancora un percorso, aumentando progressivamente la metrica. Split horizon, route poisoning e poisoned reverse riducono il problema.

```
# /etc/frr/daemons
zebra=yes
ripd=yes

# Estratto frr.conf
router rip
version 2
network 100.1.0.0/16
redistribute connected

# Annuncio di una default route (a seconda del lab/policy)
router rip
default-information originate
```

Meccanismo	Idea
Split horizon	Non annunciare una rotta sulla stessa interfaccia da cui è stata appresa.
Route poisoning	Annunciare una rotta fallita con metrica infinita.

Meccanismo	Idea
Poisoned reverse	Annunciare esplicitamente al next-hop che la rotta appresa tramite lui è infinita.
Triggered update	Inviare subito un cambiamento invece di attendere il timer periodico.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Quale algoritmo usa RIP e quale metrica adotta?
2. **[Base]** Perché il valore 16 è considerato infinito?
3. **[Intermedia]** Spiega il processo iterativo con cui un router aggiorna le distanze usando le tabelle dei vicini.
4. **[Intermedia]** Che cosa significa convergenza in un protocollo distance-vector?
5. **[Intermedia]** Come funzionano split horizon e poisoned reverse?
6. **[Difficile]** Ricostruisci un esempio di count-to-infinity con tre router dopo la caduta di un link.
7. **[Difficile]** Perché la metrica hop-count può scegliere un percorso peggiore in termini di banda o latenza?
8. **[Difficile]** Quali differenze operative esistono tra aggiornamento periodico e triggered update?
9. **[Molto difficile]** Confronta il tipo di conoscenza posseduta da un router RIP con quella di un router OSPF e collega la differenza ai loop e alla complessità.

12. OSPF I: Link State, LSDB, adiacenze, DR/BDR ed ECMP

OSPF è un IGP link-state. I router descrivono i propri collegamenti tramite LSA, li diffondono in modo affidabile e costruiscono una LSDB coerente all'interno dell'area. Ogni router esegue SPF/Dijkstra ponendo se stesso alla radice e calcola i percorsi a costo minimo.

Messaggio OSPF	Funzione
Hello	Scoperta e mantenimento dei vicini; elezione DR/BDR.
Database Description (DBD/DD)	Riassunto della LSDB durante la sincronizzazione.
Link State Request (LSR)	Richiesta degli LSA mancanti o più recenti.
Link State Update (LSU)	Trasporto degli LSA.
Link State Acknowledgment (LSAck)	Conferma per garantire flooding affidabile.

Il costo OSPF è associato alle interfacce. Tradizionalmente deriva da una reference bandwidth divisa per la banda dell'interfaccia, ma può essere impostato manualmente. Modificare un costo genera nuovi LSA e un ricalcolo SPF. Se esistono più percorsi di uguale costo, OSPF può installarli in ECMP.

```
# Configurazione essenziale
router ospf
ospf router-id 1.1.1.1
```

```

network 10.0.0.0/16 area 0.0.0.0
passive-interface eth2

interface eth0
ip ospf cost 10

# Verifica
show ip ospf
show ip ospf neighbor
show ip ospf interface
show ip ospf database
show ip ospf route
show ip route ospf

```

Su reti broadcast con più router, OSPF elegge un DR e un BDR per ridurre il numero di adiacenze e il flooding ridondante. L'elezione usa priorità di interfaccia e, in parità, Router ID. È non preemptive: l'arrivo di un router con priorità superiore non sostituisce automaticamente un DR già operativo.

Stato	Significato sintetico
Down/Attempt	Nessun Hello valido ricevuto / tentativo su reti non broadcast.
Init	Ricevuto Hello dal vicino, ma il proprio Router ID non è ancora visto nel suo Hello.
2-Way	Comunicazione bidirezionale; su broadcast può essere stato finale tra DROther.
ExStart/Exchange	Negoziazione master/slave e scambio DBD.
Loading	Richiesta degli LSA mancanti.
Full	LSDB sincronizzate tra i due adiacenti.

Le interfacce verso LAN con soli host dovrebbero essere passive: la rete resta annunciata ma non vengono inviati Hello inutili. In alternativa, se la rete non è inclusa in OSPF e viene redistribuita come connected, cambia il tipo di rotta e la semantica del dominio.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Quali sono i cinque tipi di messaggi OSPF e a cosa servono?
2. **[Base]** Che cos'è la LSDB e perché deve essere coerente tra router della stessa area?
3. **[Intermedia]** Descrivi il passaggio da Hello a stato Full.
4. **[Intermedia]** Perché su una LAN multi-accesso vengono eletti DR e BDR?
5. **[Intermedia]** Qual è la differenza tra vicino e adiacente OSPF?
6. **[Intermedia]** Perché una interfaccia passive è preferibile su una LAN di soli host?
7. **[Difficile]** Perché due DROther possono restare in 2-Way senza che ciò sia un errore?
8. **[Difficile]** Come cambia la rete quando aumenti il costo di un link? Descrivi LSA, SPF e FIB.
9. **[Difficile]** Quali mismatch impediscono la formazione di un'adiacenza OSPF?
10. **[Difficile]** Perché una perdita del 1% può essere tollerata ma perdite elevate causano flapping?

11. [Molto difficile] OSPF trova due percorsi di uguale costo. Spiega come la decisione passa da LSDB a SPF, zebra e kernel, e come viene bilanciato il traffico.

13. OSPF II: multi-area, ABR, ASBR, LSA e stub areas

La gerarchia multi-area limita la dimensione della LSDB e il dominio di flooding. L'Area 0 è il backbone logico: il traffico inter-area deve attraversarla. Un internal router appartiene a una sola area; un ABR collega aree e mantiene una LSDB separata per ciascuna; un ASBR redistribuisce rotte da altri domini o protocolli.

Tipo di LSA	Origine	Cosa descrive
Type 1 - Router LSA	Ogni router, per area	Link e stato del router dentro l'area.
Type 2 - Network LSA	DR su rete multi-accesso	Segmento condiviso e router collegati.
Type 3 - Summary LSA	ABR	Prefissi raggiungibili in altre aree.
Type 4 - ASBR Summary	ABR	Come raggiungere l'ASBR.
Type 5 - AS External	ASBR	Rotte esterne redistribute nel dominio OSPF.

```
# Esempio concettuale
router ospf
 network 10.0.0.0/24 area 0.0.0.0
 network 10.1.0.0/24 area 1.1.1.1
 area 1.1.1.1 stub

# Su tutti i router dell'area stub deve esserci configurazione compatibile.
# Variante totalmente stub (sintassi FRR/Cisco-like, lato ABR):
area 1.1.1.1 stub no-summary

# Verifica
show ip ospf database summary
show ip ospf database external
show ip ospf route
```

In una stub area gli LSA esterni Type 5 non entrano; l'ABR inietta una default route. Una totally stub area sopprime anche la maggior parte dei summary inter-area e conserva essenzialmente la default route, riducendo ulteriormente la tabella. Le rotte esterne possono apparire come E1 o E2: E1 somma il costo interno verso l'ASBR, E2 usa principalmente il costo esterno e, in parità, il costo interno come tie-break.

Coerenza di area Le opzioni stub devono essere compatibili tra i router dell'area. Un mismatch può impedire la formazione dell'adiacenza perché i bit di area/options nei messaggi Hello non coincidono.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

- [Base]** Quali ruoli hanno internal router, ABR e ASBR?
- [Base]** Perché l'Area 0 è speciale?
- [Intermedia]** Perché un ABR mantiene una LSDB separata per ogni area?

4. **[Intermedia]** Che differenza c'è tra LSA Type 1, 2, 3, 4 e 5?
5. **[Intermedia]** Come riconosci una rotta inter-area nell'output OSPF?
6. **[Difficile]** Perché una stub area blocca gli LSA Type 5 e come raggiunge comunque destinazioni esterne?
7. **[Difficile]** Che differenza c'è tra stub e totally stub?
8. **[Difficile]** Confronta E1 ed E2 in una topologia con due ASBR a distanza interna diversa.
9. **[Difficile]** Perché non esiste un'adiacenza “attraverso” un ABR tra router di aree diverse?
10. **[Molto difficile]** Descrivi come un prefisso BGP redistribuito da un ASBR raggiunge un router in una stub area e quali informazioni vengono soppresse o sostituite.

14. BGP: eBGP, iBGP, sessioni, annunci e prevenzione dei loop

BGP è un protocollo path-vector usato per il routing inter-dominio. Internet è suddivisa in Autonomous System (AS), ciascuno sotto un'unica amministrazione. eBGP scambia rotte tra AS diversi; iBGP distribuisce rotte BGP all'interno dello stesso AS. BGP usa TCP porta 179 e i peer sono configurati esplicitamente.

Stato BGP	Significato
Idle	Inizializzazione e attesa dell'evento di avvio.
Connect	Tentativo di stabilire la connessione TCP.
Active	Nuovo tentativo dopo un fallimento; non significa “sessione attiva”.
OpenSent	OPEN inviato, attesa dell'OPEN remoto.
OpenConfirm	OPEN accettato, attesa di KEEPALIVE.
Established	Sessione operativa: scambio UPDATE e KEEPALIVE.

```
# r1 in AS 100
router bgp 100
  bgp router-id 1.1.1.1
  neighbor 192.0.2.2 remote-as 200
  network 10.1.0.0/24

# Verifica
show ip bgp summary
show ip bgp neighbors
show ip bgp
show ip route bgp
```

Un peering Established non implica raggiungibilità end-to-end: i prefissi delle LAN devono essere originati o redistribuiti e devono superare le policy. In molte configurazioni FRR l'annuncio network richiede che il prefisso esista nella RIB locale; inoltre alcune versioni richiedono una policy eBGP esplicita, salvo disabilitare il controllo globale nel laboratorio.

AS_PATH elenca gli AS attraversati dall'annuncio. Quando un AS riceve un UPDATE che contiene il proprio ASN, rifiuta la rotta: è il principale meccanismo anti-loop inter-dominio. BGP non cerca soltanto la via “più veloce”: applica policy commerciali e tecniche.

Messaggio	Scopo
OPEN	Negoziare ASN, versione, hold time e capability.

Messaggio	Scopo
KEEPALIVE	Mantiene la sessione e conferma l'OPEN.
UPDATE	Annuncia NLRI/attributi o ritira prefissi.
NOTIFICATION	Segnala errore e chiude la sessione.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Qual è la differenza tra eBGP e iBGP?
2. **[Base]** Perché BGP usa TCP e su quale porta?
3. **[Base]** Che cosa mostra `show ip bgp summary`?
4. **[Intermedia]** Perché una sessione Established può non avere prefissi scambiati?
5. **[Intermedia]** Che cosa contiene un UPDATE BGP oltre al prefisso?
6. **[Intermedia]** Come AS_PATH previene i loop?
7. **[Difficile]** Perché lo stato Active spesso indica un problema e non una sessione funzionante?
8. **[Difficile]** Confronta la prevenzione dei loop in BGP con quella di OSPF e RIP.
9. **[Difficile]** Che differenza c'è tra BGP RIB, RIB globale FRR e FIB del kernel?
10. **[Molto difficile]** Descrivi tutti i passaggi necessari perché h1 in AS100 raggiunga h2 in AS200, distinguendo raggiungibilità del peering, origine dei prefissi, best-path e forwarding.

15. Policy BGP, best-path, multihoming e traffic engineering

BGP è una policy engine. Prima di accettare o esportare una rotta, il router applica filtri e route-map. Le relazioni commerciali tipiche sono provider-customer e peering; la regola “no free transit” evita che una rete cliente o un provider trasporti traffico tra terzi senza accordo.

Strumento	Cosa seleziona/modifica	Uso
Prefix-list	Prefissi IP e lunghezza	Permettere/bloccare annunci specifici.
AS-path access-list	Sequenza di ASN tramite regex	Filtrare per origine o percorso.
Route-map	Match + set	Applicare policy e modificare attributi.
Local Preference	Preferenza interna all'AS, valore alto preferito	Controllo dell'uscita (outbound).
MED	Suggerimento al vicino, valore basso preferito	Influenzare ingresso tra link verso lo stesso AS, se accettato.
AS-path prepend	Allunga artificialmente AS_PATH	Rendere meno attraente un ingresso.
Communities	Etichette di policy	Attivare comportamenti concordati con provider.


```
ip prefix-list LAN permit 203.0.113.0/24

route-map PREFER-ISP2 permit 10
  match ip address prefix-list LAN
  set local-preference 200

route-map PREPEND-ISP1 permit 10
  set as-path prepend 65003 65003 65003

router bgp 65003
  neighbor 198.51.100.1 route-map PREFER-ISP2 in
  neighbor 192.0.2.1 route-map PREPEND-ISP1 out
```

In uno stub multi-homed con ISP2 primario e ISP1 backup, una local preference maggiore sulle rotte apprese da ISP2 influenza l'uscita dall'AS. Per influenzare il traffico in ingresso, l'AS può prependerne il percorso verso ISP1. Sono strumenti euristici: gli AS remoti possono applicare proprie policy e ignorare l'intento.

Il best-path completo dipende dall'implementazione. In una spiegazione d'esame è sufficiente presentare l'ordine semplificato e dichiarare che esistono tie-break ulteriori: policy locale/weight, local preference, origine locale, AS_PATH più corto, origin, MED, preferenza eBGP/iBGP, costo IGP verso il next-hop e criteri finali. Non presentare l'AS_PATH come unica regola.

Next-hop e recursive lookup In iBGP il next-hop di una rotta eBGP può restare invariato. Se i router interni non sanno raggiungerlo tramite IGP, la rotta è inutilizzabile. next-hop-self riscrive il next-hop verso il router iBGP che annuncia la rotta.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Quale attributo useresti per preferire un ISP in uscita dal tuo AS?
2. **[Base]** Che cosa fa l'AS-path prepending?
3. **[Intermedia]** Perché local preference non può essere imposta direttamente negli AS altrui?
4. **[Intermedia]** Come si combinano prefix-list e route-map?
5. **[Intermedia]** Quale differenza c'è tra controllare traffico inbound e outbound?
6. **[Difficile]** Perché il prepending non garantisce che tutto il traffico remoto cambi percorso?
7. **[Difficile]** Spiega il problema del next-hop invariato in iBGP e il ruolo di next-hop-self.
8. **[Difficile]** Costruisci una policy “no free transit” per una rete cliente dual-homed.
9. **[Difficile]** Confronta MED, local preference e AS-path prepend per direzione, visibilità e affidabilità della policy.
10. **[Molto difficile]** Due rotte hanno local preference uguale, AS_PATH diverso e costo IGP verso il next-hop opposto. Quale criterio domina e perché il risultato può sembrare non ottimale dal punto di vista della latenza end-to-end?
11. **[Molto difficile]** Progetta una strategia di load sharing inbound con prefissi più specifici e una route aggregata di backup, spiegando l'effetto del Longest Prefix Match.

16. BGP nei data center: Clos/Fat-Tree, ECMP e failure handling

Le reti Clos/Fat-Tree offrono molti percorsi di uguale lunghezza tra server. Gli strati tipici sono Leaf, Spine e, in topologie multilivello, ToF/Top of Fabric. Nei data center moderni eBGP viene usato anche internamente perché il modello è semplice, scalabile e limita il flooding tipico dei link-state in topologie molto dense.

```
server -- Leaf1 == Spine1 == Leaf2 -- server
        \      //
         == Spine2 ==

# Più percorsi di uguale costo -> ECMP
```

Con ECMP più next-hop validi vengono installati nella FIB. Il kernel distribuisce i flussi tramite hashing, tipicamente su una 5-tuple: IP sorgente, IP destinazione, porta sorgente, porta destinazione, protocollo. Il bilanciamento per flusso evita il riordinamento che si avrebbe scegliendo casualmente per pacchetto.

```
router bgp 65001
 maximum-paths 8
 bgp bestpath as-path multipath-relax

# Peer-group per ridurre la ripetizione
neighbor SPINES peer-group
neighbor SPINES remote-as external
neighbor 10.0.0.2 peer-group SPINES
neighbor 10.0.0.6 peer-group SPINES
```

In alcune assegnazioni ASN ogni leaf ha un AS distinto, mentre gruppi di spine/ToF possono condividere ASN. multipath-relax consente di trattare come equivalenti percorsi con AS_PATH di uguale lunghezza ma sequenze di ASN diverse. La scelta dipende dal design del laboratorio.

Un guasto a un link rimuove un next-hop ECMP. La connettività può rimanere attiva mentre la capacità aggregata diminuisce: è una degradazione morbida. I timer BGP nei data center vengono spesso ridotti e BFD può fornire rilevamento sub-secondo; timer troppo aggressivi, però, aumentano il rischio di flapping e carico di controllo.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Che differenza c'è tra Leaf e Spine?
2. **[Base]** Che cosa significa ECMP?
3. **[Intermedia]** Perché un hash per flusso è preferibile a una scelta per pacchetto?
4. **[Intermedia]** A cosa serve un peer-group BGP?
5. **[Intermedia]** Perché una topologia Clos offre resilienza e alta bisection bandwidth?
6. **[Difficile]** Perché BGP può essere preferito a OSPF in un fabric molto grande?
7. **[Difficile]** Che problema risolve `bgp bestpath as-path multipath-relax`?
8. **[Difficile]** Perché traceroute classico può mostrare risultati incompleti o variabili in presenza di ECMP?
9. **[Difficile]** Spiega cosa succede a RIB, FIB e throughput quando cade uno dei quattro percorsi ECMP.

10. **[Molto difficile]** Progetta un test con iperf3 capace di mostrare che più flussi vengono distribuiti su spine diverse senza confondere il risultato con il singolo-flow hashing.

PARTE III

Misure, servizi e trasporto

Prestazioni, traffic control, load balancing, KaaS, TCP, congestione, QUIC, SDN e gestione.

Obiettivo della parte Costruire una comprensione operativa: non memorizzare soltanto i comandi, ma saper prevedere cosa cambia nelle tabelle, nei pacchetti e nello stato dei protocolli.

17. iperf3 e Linux Traffic Control

iperf3 genera traffico sintetico in modello client-server. Il server ascolta normalmente su TCP/UDP 5201; il client avvia il test e calcola throughput, trasferimento e altre statistiche. Non misura “la velocità assoluta della rete” in ogni senso: il risultato dipende da protocollo, numero di flussi, buffer, RTT, CPU e configurazione del percorso.

Comando	Scopo
iperf3 -s	Server in foreground.
iperf3 -s -D	Server in background/daemon.
iperf3 -c 10.0.0.2	Test TCP client standard.
iperf3 -c 10.0.0.2 -P 4	Quattro flussi paralleli.
iperf3 -c 10.0.0.2 -w 256K	Richiede una window/buffer specifica.
iperf3 -c 10.0.0.2 -C cubic	Seleziona algoritmo di congestione se supportato.
iperf3 -c 10.0.0.2 -u -b 8M	UDP a 8 Mbit/s target.
iperf3 -c 10.0.0.2 -J	Output JSON per automazione.

Il Bandwidth-Delay Product (BDP) è banda x RTT e rappresenta la quantità di dati in volo necessaria a riempire il percorso. Una finestra troppo piccola limita il throughput su reti ad alta banda e latenza. Con UDP le metriche fondamentali diventano perdita e jitter; il target -b deve essere scelto consapevolmente.

Linux Traffic Control (tc) manipola le code in uscita. Le qdisc decidono come accodare e servire i pacchetti; le qdisc classful possono contenere classi; i filter classificano il traffico. TBF limita la banda, netem inietta ritardo/perdita/jitter/corruzione, HTB crea gerarchie di banda, fq_codel combatte bufferbloat.

```
# Limitare la banda
tc qdisc add dev eth0 root tbf rate 10mbit burst 32kbit latency 400ms

# Inserire 100 ms di ritardo e 1% di perdita
tc qdisc add dev eth0 root netem delay 100ms loss 1%

# Vedere la configurazione
tc qdisc show dev eth0

# Pulire la qdisc root prima di un nuovo esperimento
tc qdisc del dev eth0 root
```

Riproducibilità Le regole tc sono runtime e normalmente scompaiono al riavvio. Nei laboratori documenta sempre interfaccia, qdisc, rate, delay, loss e momento di applicazione; cancella la qdisc precedente prima di sovrapporre nuovi esperimenti.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Quali ruoli hanno iperf3 -s e iperf3 -c?
2. **[Base]** Quali metriche osservi in un test UDP oltre al throughput?
3. **[Intermedia]** Perché più flussi paralleli possono ottenere più banda di un singolo flusso?
4. **[Intermedia]** Che cos'è il BDP e come si collega alla finestra TCP?
5. **[Intermedia]** Che differenza c'è tra TBF e netem?
6. **[Difficile]** Perché un test iperf in Kathara può misurare la CPU dell'host oltre alla rete emulata?
7. **[Difficile]** Come progetteresti un esperimento per separare l'effetto della perdita da quello del ritardo?
8. **[Difficile]** Perché applicare una qdisc sull'interfaccia sbagliata può produrre risultati apparentemente incoerenti?
9. **[Molto difficile]** Calcola il BDP di un percorso a 100 Mbit/s e RTT 80 ms e discuti quale finestra minima serve per saturarlo.

18. Load balancing a due livelli: DNS e Layer 4

Il laboratorio combina due meccanismi indipendenti. Il DNS restituisce più indirizzi per lo stesso nome, distribuendo i client tra load balancer front-end. Ogni load balancer L4 distribuisce poi le connessioni TCP verso un pool di web server. Il primo livello agisce sulla scelta dell'endpoint; il secondo sul forwarding delle connessioni.

```
# Zona DNS: più record A per lo stesso nome
www.uniroma3.it. 60 IN A 203.0.113.10
www.uniroma3.it. 60 IN A 203.0.113.20

# Il client può osservare le risposte
dig www.uniroma3.it

# Test applicativo
links http://www.uniroma3.it
curl -v http://www.uniroma3.it
```

Il DNS round-robin non conosce necessariamente lo stato dei server e i client/cache possono mantenere un IP fino alla scadenza del TTL. Un guasto a un load balancer può quindi continuare a colpire utenti che hanno una risposta in cache. Il load balancer L4 può invece eseguire health check sui backend e rimuovere rapidamente un web server guasto dal pool.

Guasto	Senza health/TTL breve	Con mitigazione
Web server backend	Alcune connessioni falliscono o vanno in timeout.	Health check rimuove il backend dal pool.
Load balancer	Client con IP in cache continuano a tentare il nodo guasto.	Riduzione TTL, DNS health-aware, anycast o VIP ridondante.
DNS autoritativo	Nuove risoluzioni possono fallire.	Ridondanza autoritativa e caching distribuito.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Qual è la differenza tra DNS load balancing e L4 load balancing?
2. **[Base]** Perché più record A possono distribuire i client?

3. **[Intermedia]** Quale ruolo ha il TTL durante un guasto?
4. **[Intermedia]** Che cosa controlla un health check di backend?
5. **[Intermedia]** Perché il round-robin DNS non garantisce distribuzione perfettamente uniforme?
6. **[Difficile]** Che cosa accade alle connessioni già aperte quando un backend viene rimosso dal pool?
7. **[Difficile]** Confronta failure detection al livello DNS e al livello L4.
8. **[Difficile]** Come dimostreresti con Wireshark che il client ha ricevuto più IP e poi ha aperto una connessione verso uno specifico LB?
9. **[Molto difficile]** Progetta una strategia per ridurre il downtime senza impostare un TTL irrealisticamente basso, spiegando costi e trade-off.

19. KaaS: Kathara as a Service

KaaS rende l'emulazione accessibile tramite browser. L'utente disegna o carica la topologia; il frontend comunica con una REST API; i laboratori vengono eseguiti in un ambiente Docker-in-Docker isolato; un terminale web espone le shell dei device. Traefik funge da reverse proxy e instrada le richieste verso i servizi interni.



Componente	Responsabilità
Traefik	Reverse proxy, routing per hostname e ingresso unificato.
Docker-in-Docker	Ambiente isolato in cui vengono creati i container dei laboratori.
Web Interface	Editor visuale e gestione del ciclo di vita del lab.
REST API	Traduzione delle operazioni utente in comandi/azioni backend.
Web TTY	Accesso interattivo alle shell tramite canale realtime.

L'isolamento è un tema centrale: eseguire container dentro un ambiente dedicato riduce l'esposizione diretta del Docker daemon dell'host, ma Docker-in-Docker aggiunge complessità, storage annidato e requisiti di sicurezza. In un sistema multiutente servono limiti di CPU/RAM, autenticazione, quote e separazione dei lab.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Quali sono i principali componenti di KaaS?
2. **[Base]** Che ruolo ha Traefik?
3. **[Intermedia]** Perché usare Docker-in-Docker invece di creare direttamente i lab sul Docker dell'host?

4. **[Intermedia]** Quale differenza c'è tra REST API e canale Web TTY/socket.io?
5. **[Difficile]** Quali risorse devono essere limitate in un servizio KaaS multiutente?
6. **[Difficile]** Quali nuovi failure domain introduce l'architettura a microservizi?
7. **[Difficile]** Come tradurresti un grafo drag-and-drop in righe lab.conf evitando collision domain incoerenti?
8. **[Molto difficile]** Analizza i rischi di sicurezza di esporre terminali dei container via browser e proponi controlli a più livelli.

20. Livello di trasporto, RDT e fondamentali di TCP

Il livello di trasporto offre multiplexing/demultiplexing tramite porte e, nel caso TCP, affidabilità sopra una rete IP non affidabile. Un socket UDP è identificato essenzialmente da IP e porta locali per la consegna; una connessione TCP è identificata dalla 4-tuple IP/porta locale e IP/porta remota.

Per costruire un trasferimento affidabile servono checksum, feedback, numeri di sequenza, timer e ritrasmissioni. Stop-and-wait è corretto ma inefficiente su percorsi con BDP elevato. Il pipelining consente più pacchetti in volo; Go-Back-N usa ACK cumulativi e ritrasmette dal pacchetto mancante in avanti, mentre Selective Repeat conserva i pacchetti fuori ordine e ritrasmette solo i mancanti.

Meccanismo	Problema risolto
Checksum	Rilevazione di corruzione.
Sequence number	Ordine, duplicati e identificazione dei dati.
ACK	Conferma della ricezione.
Timeout/RTO	Perdita di dati o ACK.
Finestra	Pipelining e utilizzo del percorso.
Buffer ricevente	Riordinamento e flow control.

TCP aggiunge three-way handshake, chiusura, byte sequence numbers, ACK cumulativi, flow control tramite rwnd e congestion control tramite cwnd. La quantità effettivamente inviabile è limitata dal minimo tra finestra del ricevente e finestra di congestione, oltre ai vincoli applicativi e del sistema.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Qual è la differenza tra multiplexing e demultiplexing?
2. **[Base]** Da quali campi è identificata una connessione TCP?
3. **[Intermedia]** Perché checksum e ACK da soli non bastano a gestire la perdita totale di un pacchetto?
4. **[Intermedia]** A cosa servono i numeri di sequenza in presenza di ACK corrotti o ritrasmissioni?
5. **[Intermedia]** Perché stop-and-wait utilizza male un link ad alto BDP?
6. **[Difficile]** Confronta Go-Back-N e Selective Repeat per buffer, ACK e quantità di ritrasmissioni.
7. **[Difficile]** Qual è la differenza tra flow control e congestion control?
8. **[Difficile]** Perché TCP numera byte e non semplicemente segmenti?
9. **[Molto difficile]** Descrivi come un timeout può essere ambiguo: il dato o l'ACK potrebbero essere persi. Come i numeri di sequenza rendono il protocollo corretto?

21. Controllo di congestione, fairness, bufferbloat, AQM ed ECN

Il congestion control evita il collasso causato da sorgenti che inviano più dati di quanti la rete possa trasportare. TCP usa feedback implicito: ACK regolari indicano capacità disponibile; perdita, ECN o aumento di ritardo segnalano congestione. cwnd controlla i dati in volo dal lato mittente.

Fase/evento	Comportamento classico Reno/Tahoe
Slow Start	cwnd cresce esponenzialmente, circa raddoppiando per RTT.
Congestion Avoidance	Crescita additiva, circa 1 MSS per RTT.
Timeout	Segnale grave: riduzione forte, ssthresh aggiornata e ripartenza prudente.
3 duplicate ACK	Fast retransmit; Reno usa fast recovery evitando una ripartenza completa.

AIMD - Additive Increase, Multiplicative Decrease - converge verso una condivisione efficiente e relativamente equa: aumento additivo in assenza di congestione, riduzione moltiplicativa quando la rete segnala sovraccarico. RTT diversi possono introdurre bias: flussi con RTT breve aggiornano più spesso la finestra.

TCP CUBIC usa una funzione cubica del tempo dall'ultima congestione, meno dipendente dall'RTT e adatta a link veloci e lunghi. DCTCP usa marcature ECN e una riduzione proporzionale per mantenere code corte nei data center. fq_codel combina fair queuing e controllo del ritardo per limitare il bufferbloat.

Tema	Idea
Max-Min fairness	Massimizzare il flusso più piccolo senza ridurre flussi già minori o uguali.
Proportional fairness	Massimizzare la somma dei logaritmi delle velocità, bilanciando efficienza ed equità.
Bufferbloat	Buffer eccessivi mantengono throughput ma generano latenza enorme.
AQM/RED	Marcare/scartare prima della saturazione completa usando una misura della coda.
ECN	Marcare congestione senza scartare il pacchetto; il ricevente informa il mittente.
TCP-friendly	Un flusso non TCP deve reagire alla congestione in modo compatibile con i flussi TCP.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Che differenza c'è tra cwnd e rwnd?
2. **[Base]** Come crescono cwnd in Slow Start e Congestion Avoidance?
3. **[Intermedia]** Perché un timeout è considerato un segnale più grave di tre ACK duplicati?
4. **[Intermedia]** Che cosa significa AIMD e perché l'aumento è additivo mentre la riduzione è moltiplicativa?
5. **[Intermedia]** Che cos'è il bufferbloat?
6. **[Difficile]** Perché CUBIC è più adatto di Reno alle long fat networks?
7. **[Difficile]** Come ECN può ridurre la perdita pur segnalando congestione?
8. **[Difficile]** Confronta tail drop, RED e fq_codel.
9. **[Difficile]** Perché flussi con RTT diverso possono ottenere quote diverse anche usando lo stesso algoritmo?
10. **[Molto difficile]** Spiega il compromesso tra efficienza e fairness e perché massimizzare solo il throughput totale può essere ingiusto.

22. QUIC: HTTP/3, TLS integrato, stream e laboratorio QLOG

QUIC è un protocollo di trasporto in user space costruito sopra UDP e usato da HTTP/3. Integra TLS 1.3, riduce i round trip di setup, supporta stream indipendenti e identifica la connessione con Connection ID, facilitando la migrazione tra reti quando cambia l'indirizzo IP.

Caratteristica	Vantaggio	Precisazione
Handshake integrato	Meno RTT per connessione e cifratura	0-RTT richiede cautela per replay e dati idempotenti.
Stream indipendenti	Perdita in uno stream non blocca la consegna degli altri	La congestione a livello connessione resta condivisa.
TLS 1.3 obbligatorio	Nessuna modalità in chiaro	Gran parte degli header è protetta.
Connection ID	Migrazione Wi-Fi/4G senza perdere la sessione	Serve mantenere stato e validare il nuovo path.
User space	Aggiornamento rapido senza modificare il kernel	Più responsabilità a librerie e applicazioni.

Nel laboratorio due client condividono un bottleneck. QLOG registra eventi interni di QUIC, come cwnd, bytes in flight, RTT, perdita e recovery; script Python estraggono serie temporali e generano grafici. Per decifrare QUIC in Wireshark si esportano le chiavi TLS con un keylog file e si configura Wireshark affinché lo legga.

```
# Schema di laboratorio
client1 ----\
               router --[ bottleneck 8 Mbit/s ]-- server
client2 ----/

# Esempio di limitazione
tc qdisc add dev ethX root tbf rate 8mbit burst 32kbit latency 400ms

# File condivisi
/shared/keys.txt
/shared/*.qlog
/shared/results/*.json
```

Per valutare fairness non basta guardare il throughput finale: bisogna osservare l'evoluzione temporale, la stabilità, il tempo di convergenza, la sensibilità all'RTT e la reazione alle perdite.

Confrontare TCP, UDP non controllato e QUIC significa distinguere protocollo di trasporto, algoritmo di congestione e implementazione.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Perché QUIC è costruito sopra UDP?
2. **[Base]** Qual è la relazione tra QUIC e HTTP/3?
3. **[Intermedia]** Come QUIC riduce l'head-of-line blocking rispetto a HTTP/2 su TCP?
4. **[Intermedia]** A cosa serve il Connection ID durante il cambio di rete?
5. **[Intermedia]** Perché Wireshark non può decifrare QUIC senza chiavi di sessione?
6. **[Difficile]** Qual è la differenza tra indipendenza degli stream e condivisione del congestion controller?
7. **[Difficile]** Quali rischi introduce 0-RTT e perché non è equivalente a “nessuna sicurezza”?
8. **[Difficile]** Quali variabili QLOG useresti per spiegare una caduta di throughput?
9. **[Difficile]** Come progetteresti un confronto equo tra TCP CUBIC e QUIC, controllando RTT, loss, buffer e numero di flussi?
10. **[Molto difficile]** Due flussi QUIC con RTT diversi condividono un collo di bottiglia. Spiega quali grafici produrresti e come distingueresti unfairness strutturale da rumore dell'emulazione.

23. SDN e gestione della rete: SNMP, NETCONF e YANG

SDN separa il control plane dal data plane e concentra la logica in un controller logicamente centralizzato. Gli switch eseguono regole; il controller mantiene una visione della topologia e programma i dispositivi tramite API southbound, mentre applicazioni e orchestratori usano API northbound. La centralizzazione logica non implica necessariamente un singolo server fisico.

Tecnologia	Modello	Uso
SNMP	Manager-agent, MIB, polling e trap	Monitoraggio e gestione tradizionale.
YANG	Modello dati gerarchico	Descrive configurazione e stato in modo machine-readable.
NETCONF	Protocollo RPC su trasporto sicuro, tipicamente SSH	Legge/applica configurazioni strutturate.
RESTCONF	Accesso REST a dati modellati YANG	Integrazione con automazione web/API.
OpenFlow (esempio SDN)	Regole di flow programmate dal controller	Controllo del forwarding negli switch.

La gestione moderna mira a configurazioni dichiarative, validabili e transazionali. Rispetto alla CLI imperativa, un modello YANG consente di sapere quali dati sono ammessi, applicare controlli e automatizzare migliaia di apparati. In un orale è utile collegare SDN a traffic engineering e KaaS: entrambi separano interfaccia/controllo dall'esecuzione sottostante.

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere

definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

1. **[Base]** Che differenza c'è tra data plane e control plane in SDN?
2. **[Base]** Quali ruoli hanno northbound e southbound API?
3. **[Intermedia]** Che differenza c'è tra controller logicamente centralizzato e singolo punto fisico?
4. **[Intermedia]** Quali ruoli hanno MIB, polling e trap in SNMP?
5. **[Intermedia]** Che rapporto c'è tra YANG e NETCONF?
6. **[Difficile]** Perché una configurazione strutturata e transazionale è più adatta all'automazione della CLI testuale?
7. **[Difficile]** Confronta routing distribuito tradizionale e SDN per reazione ai guasti e consistenza dello stato.
8. **[Molto difficile]** Progetta un'architettura in cui una UI tipo KaaS genera una topologia, un controller calcola le policy e NETCONF configura i nodi: indica API, modelli e failure domain.

PARTE IV

Laboratori, procedure e preparazione all'orale

Playbook operativi, risultati attesi, errori tipici e simulazioni finali.

Obiettivo della parte Costruire una comprensione operativa: non memorizzare soltanto i comandi, ma saper prevedere cosa cambia nelle tabelle, nei pacchetti e nello stato dei protocolli.

24. Playbook dei laboratori del corso

Per ogni laboratorio usa sempre lo stesso ciclo: ispezione dei file -> dry-run -> avvio -> fotografia dello stato iniziale -> configurazione -> test -> guasto controllato -> raccolta evidenze -> pulizia. Non limitarti a ottenere un ping: devi saper spiegare quale tabella ha reso possibile ogni hop.

LAB-K1 - Docker hands-on

Obiettivo: Comprendere immagini, container, isolamento e ciclo di vita.

Sequenza minima di comandi:

```
docker run hello-world
docker pull kathara/base
docker images
docker ps -a
docker container prune
docker system df
```

Cosa devi osservare e saper spiegare:

- Spiegare perché un container terminato resta visibile con ps -a.
- Distinguere pulizia dei container e pulizia delle immagini.
- Collegare Docker al modello device/collision domain di Kathara.

Errori tipici:

- Confondere immagine e container.
- Usare prune senza controllare ciò che verrà eliminato.

LAB-K2 - Basics on Kathara

Obiettivo: Avviare due host sullo stesso collision domain e comprendere lab.conf, startup e /shared.

Sequenza minima di comandi:

```
kathara lstart --print
kathara lstart --noterminals
kathara linfo -t
kathara exec pc1 "ip -br addr"
kathara exec pc1 "ip route"
kathara exec pc1 "ping -c 3 10.0.0.2"
kathara lclean
```

Cosa devi osservare e saper spiegare:

- Prima dello startup le interfacce esistono ma gli IP possono mancare.
- L'assegnazione /24 crea una rotta connessa.
- Il primo ping può includere il costo della risoluzione ARP.

Errori tipici:

- Aspettarsi che lab.conf assegni gli indirizzi.
- Interpretare piccole variazioni RTT come proprietà della rete fisica.

LAB-K3a - Topologie e routing statico

Obiettivo: Configurare gateway, VLSM, rotte statiche, percorsi asimmetrici, traceroute e iperf3.

Sequenza minima di comandi:

```
ip addr add ...
ip route add default via ...
```

```
sysctl -w net.ipv4.ip_forward=1
ip route get DEST
traceroute -n DEST
iperf3 -s
iperf3 -c SERVER
```

Cosa devi osservare e saper spiegare:

- Verificare sempre il ritorno.
- Confrontare traceroute nei due sensi.
- Calcolare le subnet prima di scrivere gli startup.
- Mostrare un loop controllato e l'esaurimento del TTL.

Errori tipici:

- Sovrapporre subnet.
- Usare come gateway un indirizzo non on-link.
- Dimenticare ip_forward.

LAB-K4 - BGP

Obiettivo: Stabilire eBGP, originare prefissi e analizzare AS_PATH e anti-loop.

Sequenza minima di comandi:

```
vtysh -c "show ip bgp summary"
vtysh -c "show ip bgp"
vtysh -c "show ip route bgp"
ss -lnt | grep 179
tcpdump -ni ethX tcp port 179
```

Cosa devi osservare e saper spiegare:

- Separare “peering Established” da “prefissi annunciati”.
- Osservare UPDATE e withdraw.
- Verificare che la rete originata esista nella RIB.

Errori tipici:

- remote-as errato.
- network statement senza rotta locale.
- policy eBGP mancante o filtro implicito.

LAB-K5 - BGP Multi-Homed

Obiettivo: Realizzare primario/backup con local preference e AS-path prepend e misurare la convergenza.

Sequenza minima di comandi:

```
show ip bgp
show ip bgp neighbors
neighbor X shutdown
no neighbor X shutdown
ip link set ethX down
ping -i 0.2 DEST
```

Cosa devi osservare e saper spiegare:

- Local preference influenza l'uscita.
- Prepending influenza in modo non garantito l'ingresso.
- Confrontare shutdown BGP controllato e guasto fisico/timer.

Errori tipici:

- Applicare route-map nella direzione sbagliata.
- Confondere inbound dal punto di vista del router con traffico dati inbound dell'AS.

LAB-K6 - BGP in a data center

Obiettivo: Comprendere Clos, peer-group, ECMP, hashing e degradazione morbida.

Sequenza minima di comandi:

```
show ip bgp
show ip route
ip route
iperf3 -P 8
ip link set ethX down
kathara linfo -w
```

Cosa devi osservare e saper spiegare:

- Contare i next-hop installati.
- Generare più flussi per attivare hash diversi.
- Misurare continuità e riduzione di capacità dopo un guasto.

Errori tipici:

- Concludere che ECMP non funziona usando un solo flusso.
- Usare traceroute classico come prova definitiva dei path multipli.

LAB-K7 - Load balancing a due livelli

Obiettivo: Analizzare round-robin DNS, load balancing L4, health check e TTL.

Sequenza minima di comandi:

```
dig www.uniroma3.it
tcpdump -ni any port 53
links http://www.uniroma3.it
curl -v http://www.uniroma3.it
ss -lntup
```

Cosa devi osservare e saper spiegare:

- Distinguere IP scelto dal DNS e backend scelto dal LB.
- Spegnere un web server e osservare health check.
- Rimuovere un LB dal DNS e osservare l'effetto della cache.

Errori tipici:

- Aspettarsi failover immediato nei client con cache.
- Attribuire al DNS la scelta del backend interno.

LAB-KAAS - Kathara as a Service

Obiettivo: Comprendere la catena UI/API/Traefik/DinD/TTY e i problemi di isolamento.

Sequenza minima di comandi:

```
docker compose ps
docker compose logs
curl API_ENDPOINT
docker stats
```

Cosa devi osservare e saper spiegare:

- Seguire una richiesta dalla UI fino alla creazione del lab.
- Identificare dove avvengono autenticazione, routing HTTP e accesso TTY.
- Valutare limiti e sicurezza multiutente.

Errori tipici:

- Esporre il Docker socket host senza controllo.
- Confondere reverse proxy e load balancer di rete emulata.

LAB OSPF I - Single Area

Obiettivo: Formare adiacenze, leggere LSDB, cambiare costi, osservare DR/BDR e perdita.

Sequenza minima di comandi:

```
show ip ospf neighbor
show ip ospf database
show ip ospf interface
show ip ospf route
tc qdisc add dev ethX root netem loss 1%
```

Cosa devi osservare e saper spiegare:

- Annotare stati e ruoli DR/BDR.
- Cambiare un costo e confrontare route prima/dopo.
- Aumentare loss gradualmente e osservare flapping.

Errori tipici:

- Interpretare 2-Way tra DROther come errore.
- Applicare netem sull'interfaccia non attraversata dagli Hello.

LAB OSPF II - Multi-Area

Obiettivo: Analizzare ABR, summary LSA, stub/totally stub, default route e ridistribuzione esterna.

Sequenza minima di comandi:

```
show ip ospf database summary
show ip ospf database external
show ip ospf route
show ip route ospf
```

Cosa devi osservare e saper spiegare:

- Confrontare LSDB di internal router e ABR.
- Verificare O IA, O E1/O E2 e default route.
- Rendere l'area stub e poi totally stub, confrontando database e tabella.

Errori tipici:

- Configurazione stub non coerente sui router.
- Confondere Type 3 e Type 5.

LAB QUIC e Congestion Control

Obiettivo: Misurare competizione su bottleneck, fairness, perdita/RTT e analizzare QLOG/Wireshark.

Sequenza minima di comandi:

```
tc qdisc show
iperf3 / client QUIC
tcpdump -w /shared/capture.pcap
ls /shared/*.qlog
```

```
python scripts/plot.py
```

Cosa devi osservare e saper spiegare:

- Controllare rate effettivo del bottleneck.
- Sincronizzare l'avvio dei flussi.
- Confrontare throughput, cwnd, RTT, loss e tempo di convergenza.
- Importare il keylog TLS in Wireshark.

Errori tipici:

- Confrontare run con parametri diversi.
- Giudicare fairness da una sola media finale.
- Dimenticare di rimuovere qdisc precedenti.

25. Matrice di troubleshooting per i laboratori

Sintomo	Verifica 1	Verifica 2	Correzione probabile
Il device non parte	kathara lstart --print	kathara check / Docker attivo	Correggere lab.conf, image o backend.
Startup non applicato	kathara connect -l DEVICE	Controllare path/nome file	Correggere shell, sintassi o ordine.
Interfaccia mancante	kathara linfo -t	ip -br link	Correggere indice o collision domain.
IP presente, gateway non pingabile	ip neigh	tcpdump arp	Prefisso errato, dominio errato, interfaccia down.
Gateway pingabile, remoto no	ip route get	ip_forward e return route	Aggiungere rotta/forwarding.
OSPF nessun vicino	show ip ospf interface	tcpdump proto 89	Area/mask/timer/auth/passive mismatch.
BGP Active	ping neighbor	ss/tcpdump port 179	IP/AS/source/firewall errati.
BGP no prefissi	show ip bgp summary	show running-config	Origine, filtri, policy, import-check.
ECMP un solo path usato	ip route	Numero flussi/5-tuple	Abilitare multipath e generare flussi distinti.
QUIC non decifrabile	Verificare keys.txt	Preferenze TLS Wireshark	Importare keylog corretto della stessa sessione.

26. Metodo per rispondere bene all'orale

Una risposta tecnica convincente non è una definizione isolata. Usa lo schema D-M-S-V-G: Definizione, Meccanismo, Stato, Verifica, Guasto. In questo modo mostri comprensione e colleghi teoria e laboratorio.

Passo	Domanda guida	Esempio BGP
D - Definizione	Che cos'è e a quale problema risponde?	Protocollo path-vector inter-AS.
M - Meccanismo	Quali messaggi e regole usa?	TCP 179, OPEN/KEEPALIVE/UPDATE, policy.

Passo	Domanda guida	Esempio BGP
S - Stato	Quali tabelle/attributi mantiene?	BGP RIB, AS_PATH, local preference, next-hop.
V - Verifica	Quale comando dimostra ciò che dici?	show ip bgp summary; show ip bgp.
G - Guasto	Come reagisce a link o peer failure?	Withdraw, timer/BFD, nuovo best-path.

Allenamento Cronometrati: 30 secondi per una definizione, 2 minuti per una spiegazione completa, 4 minuti per una domanda con esempio e guasto. Disegna sempre una topologia minima quando la risposta coinvolge più router.

27. Simulazioni di orale integrate

Domande di autovalutazione

Metodo Rispondi ad alta voce senza consultare gli appunti. Una risposta completa deve includere definizione, livello/protocollo coinvolto, meccanismo interno, comando di verifica e comportamento in caso di guasto.

- [Molto difficile]** Partendo da una cartella Kathara vuota, progetta un laboratorio con due LAN, due router e routing OSPF. Descrivi file, collision domain, startup, configurazione FRR, comandi di verifica e risultato atteso.
- [Molto difficile]** Confronta routing statico, RIP, OSPF e BGP sulla stessa topologia: conoscenza posseduta, algoritmo, metrica/policy, loop prevention, convergenza e scala.
- [Molto difficile]** Un ping end-to-end fallisce dopo un guasto. Costruisci una diagnosi che distingua link, ARP/NDP, rotta connessa, rotta dinamica, forwarding e ritorno.
- [Molto difficile]** Spiega come una rotta BGP appresa via eBGP entra nella BGP RIB, viene selezionata, risolta ricorsivamente, inserita nella FIB e infine usata per inoltrare un pacchetto.
- [Molto difficile]** In un AS multi-homed vuoi ISP2 primario in entrambe le direzioni. Descrivi local preference, prepending, MED, prefissi più specifici e limiti di ciascuna tecnica.
- [Molto difficile]** Un fabric Clos ha quattro percorsi ECMP. Descrivi come verificare multipath nella RIB/FIB, come generare traffico che usi più path e cosa accade quando cade uno spine.
- [Molto difficile]** Confronta OSPF single-area e multi-area dal punto di vista di LSDB, flooding, SPF, ABR, summary e failure domain.
- [Molto difficile]** Spiega perché una stub area può raggiungere una rotta BGP esterna senza riceverne il Type 5, indicando il ruolo della default route.
- [Molto difficile]** Collega BDP, finestra TCP, Slow Start, CUBIC e bufferbloat in un unico ragionamento su un link a 1 Gbit/s con 100 ms RTT.
- [Molto difficile]** Progetta un esperimento Kathara per confrontare la fairness di due flussi con RTT diversi, indicando tc, generatori di traffico, metriche e possibili fonti di errore.
- [Molto difficile]** Spiega QUIC a partire dai limiti dell'evoluzione di TCP nel kernel, includendo UDP, TLS 1.3, stream, Connection ID, QLOG e analisi Wireshark.
- [Molto difficile]** Confronta DNS load balancing, L4 load balancing, ECMP e BGP traffic engineering: livello, unità di bilanciamento, stato, failover e controllo dell'operatore.
- [Molto difficile]** Descrivi come KaaS trasforma una topologia visuale in un laboratorio eseguibile e quali controlli servono per sicurezza, isolamento e quote.

14. **[Molto difficile]** Confronta control plane distribuito e SDN usando OSPF e un controller come esempi, discutendo consistenza, scala e reazione ai guasti.
15. **[Molto difficile]** Durante l'orale il docente mostra `show ip bgp` con due path. Spiega come leggeresti ogni colonna/attributo e come prevederesti la best path senza affidarti a una sola regola.

PARTE V

Appendici operative

Cheat sheet, template pronti e checklist finali.

Obiettivo della parte Costruire una comprensione operativa: non memorizzare soltanto i comandi, ma saper prevedere cosa cambia nelle tabelle, nei pacchetti e nello stato dei protocolli.

Appendice A - Cheat sheet Kathara e Linux

Categoria	Comandi essenziali
Avvio/pulizia	kathara lstart --print; kathara lstart --noterminals; kathara lclean; kathara wipe -f
Stato/accesso	kathara linfo -t; kathara linfo -w; kathara list; kathara connect -l DEVICE; kathara exec --wait DEVICE CMD
Interfacce/IP	ip -br link; ip -br addr; ip addr add; ip link set up/down
Routing	ip route; ip route get DEST; ip route add default via GW; sysctl net.ipv4.ip_forward
Neighbor	ip neigh show; ip neigh flush all; tcpdump -ni eth0 arp
Test	ping -c; ping -6 -c; traceroute -n; ss -lntup; tcpdump -ni any
Prestazioni	iperf3 -s; iperf3 -c; -P; -u -b; -j
Traffic control	tc qdisc show; tc qdisc add ... netem; tc qdisc add ... tbf; tc qdisc del dev IF root
FRR generico	vytysh; show running-config; show ip route; configure terminal; no ...; write memory
OSPF	show ip ospf neighbor/interface/database/route
BGP	show ip bgp summary; show ip bgp; show ip bgp neighbors; show ip route bgp

Appendice B - Template minimo di laboratorio

```
# lab.conf
LAB_NAME="Template"
LAB_DESCRIPTION="Due host collegati da un router"

ha[0]="LAN_A"
r1[0]="LAN_A"
r1[1]="LAN_B"
hb[0]="LAN_B"

# ha.startup
ip link set eth0 up
ip addr add 192.168.1.1/24 dev eth0
ip route add default via 192.168.1.254

# r1.startup
ip link set eth0 up
ip link set eth1 up
ip addr add 192.168.1.254/24 dev eth0
ip addr add 192.168.2.254/24 dev eth1
sysctl -w net.ipv4.ip_forward=1

# hb.startup
ip link set eth0 up
ip addr add 192.168.2.1/24 dev eth0
ip route add default via 192.168.2.254
```

Appendice C - Template FRR

I template seguenti sono volutamente minimi. Adattare indirizzi, aree, ASN, policy e sintassi alla versione FRR usata nel laboratorio.

Template OSPF

```
# /etc/frr/daemons - OSPF
zebra=yes
ospfd=yes

# /etc/frr/frr.conf - OSPF
frr defaults traditional
hostname r1
service integrated-vtysh-config
!
interface eth0
 ip ospf cost 10
!
router ospf
 ospf router-id 1.1.1.1
 network 10.0.0.0/24 area 0.0.0.0
!
line vty
```

Template BGP

```
# /etc/frr/daemons - BGP
zebra=yes
bgpd=yes

# /etc/frr/frr.conf - BGP
frr defaults traditional
hostname r1
service integrated-vtysh-config
!
router bgp 65001
 bgp router-id 1.1.1.1
 neighbor 192.0.2.2 remote-as 65002
 network 10.1.0.0/24
!
line vty
```

Template RIP

```
# /etc/frr/daemons - RIP
zebra=yes
ripd=yes

# /etc/frr/frr.conf - RIP
frr defaults traditional
hostname r1
service integrated-vtysh-config
!
router rip
 version 2
 network 10.0.0.0/8
 passive-interface eth2
!
line vty
```

Appendice D - Checklist prima dell'esame

- ☐ Riesco a spiegare container vs VM e immagine vs container senza esempi vaghi.
- ☐ So creare da zero lab.conf, .startup e directory per configurazioni FRR.
- ☐ So usare lstart, linfo, list, connect, exec, lclean, wipe e check.
- ☐ So leggere ip addr, ip route, ip route get e ip neigh.

- ☐ So spiegare inoltro diretto/indiretto, default route e Longest Prefix Match.
- ☐ So calcolare subnet /22, /25, /30 e motivare una VLSM.
- ☐ So spiegare ARP Request/Reply, cache, gratuitous/proxy ARP e NDP.
- ☐ So configurare e diagnosticare routing statico e un routing loop.
- ☐ So spiegare Bellman-Ford, count-to-infinity e poisoned reverse.
- ☐ So descrivere LSDB, LSA, SPF, stati OSPF, DR/BDR e passive interface.
- ☐ So distinguere ABR, ASBR, LSA 1-5, O IA, E1/E2, stub e totally stub.
- ☐ So configurare un peering BGP, originare un prefisso e leggere show ip bgp.
- ☐ So spiegare local preference, MED, prepend, route-map, prefix-list e next-hop-self.
- ☐ So spiegare Clos/Fat-Tree, ECMP, hashing 5-tuple e failure degradation.
- ☐ So usare iperf3 e tc in modo riproducibile e interpretare BDP, loss e jitter.
- ☐ So distinguere DNS LB, L4 LB, health check e problema del TTL.
- ☐ So spiegare affidabilità, pipeline, GBN/SR, flow control e congestion control.
- ☐ So descrivere Slow Start, AIMD, Reno, CUBIC, AQM, ECN e fairness.
- ☐ So spiegare QUIC, HTTP/3, TLS 1.3, stream, Connection ID, QLOG e keylog.
- ☐ So collegare SDN, SNMP, NETCONF/YANG e KaaS alla separazione dei piani.

Fonti e note di accuratezza

Questo manuale è stato costruito integrando il materiale fornito nel corso, la relazione dei laboratori e la documentazione ufficiale di Kathara consultata nel luglio 2026. I comandi di protocolli e daemon possono variare leggermente in base alla versione dell'immagine FRR e alle convenzioni del laboratorio; verificare sempre con “?” in vtysh, con show running-config e con le man page installate.

- Laboratorio completi.pdf - relazione dei laboratori di Complementi di Reti 2025-2026.
- Appunti completi complementi di reti.pdf - appunti teorici su routing, OSPF, BGP, trasporto, congestione e QUIC.
- Kathara official manual pages - kathara, lab.conf, lab.dep, lab.ext, lab directory structure, lstart, lclean, linfo, list, connect, exec, settings, check e wipe.
- Documentazione e CLI FRRouting disponibile nell'immagine usata dai laboratori.

Nota finale Il documento non include le soluzioni alle domande di autovalutazione: lo scopo è costringere il richiamo attivo. Dopo ogni risposta, verifica i punti essenziali nel capitolo e ripeti la domanda finché riesci a collegare teoria, comando e comportamento osservabile.